Smart Notes
Aplicación de Notas Inteligente con IA

Aaron Rodrigo Ramos Reyes
Universidad Autónoma Metropolitana - Unidad Cuajimalpa

27 de marzo de 2026

Índice

1 Introducción **3**

2 Tecnologías utilizadas **3**

3 Arquitectura del proyecto **3**
3.1 Diagrama de clases 4
3.2 Diagrama de flujo (agregar nota) 5

4 Integración con OpenRouter (IA) **5**
4.1 Cómo funciona la integración: 5

5 Pruebas Realizadas con la IA **6**

6 Configuración Manual de Alarmas **8**

7 Heurísticas de Nielsen: Evaluación del Proyecto **9**

8 Metas Cumplidas y Comparación **9**

9 Áreas que Quedaron Pendientes y Mejoras Futuras **10**

10 Conclusión y Aprendizaje **10**

1. Introducción

Smart Notes es una aplicación móvil desarrollada en Flutter que permite gestionar notas personales organizadas en grupos (carpetas). El proyecto implementa funcionalidades de arrastrar y soltar, personalización de colores, historial de edición (Undo/Redo) y persistencia local mediante Hive.

La característica principal es su **asistente IA integrado**, que analiza automáticamente el contenido de las notas y sugiere acciones útiles como configurar recordatorios automáticos (alarmas) y crear o mover notas a grupos temáticos.

La aplicación cuenta con un diseño minimalista, sistema de grupos, arrastrar y soltar, y notificaciones locales.

2. Tecnologías utilizadas

- **Lenguaje:** Dart 3.x
- **Framework:** Flutter 3.x
- **Persistencia local:** Hive + Hive Flutter (NoSQL, rápido y nativo)
- **Generación de código:** build_runner, hive_generator
- **Notificaciones:** flutter_local_notifications
- **Inteligencia Artificial:** OpenRouter
- **IDE:** Visual Studio Code
- **Control de versiones:** Git

3. Arquitectura del proyecto

La aplicación sigue una arquitectura basada en capas simplificada:

- **Capa de presentación:** Widgets (screens, diálogos, widgets reutilizables).
- **Capa de lógica de negocio:** Servicios (`AIService`, `IntelligentAssistant`) y manejo de estado con `ValueNotifier` + `ValueListenableBuilder`.
- **Capa de datos:** Modelos (`Note`, `NoteGroup`) y adaptadores Hive.

3.1. Diagrama de clases

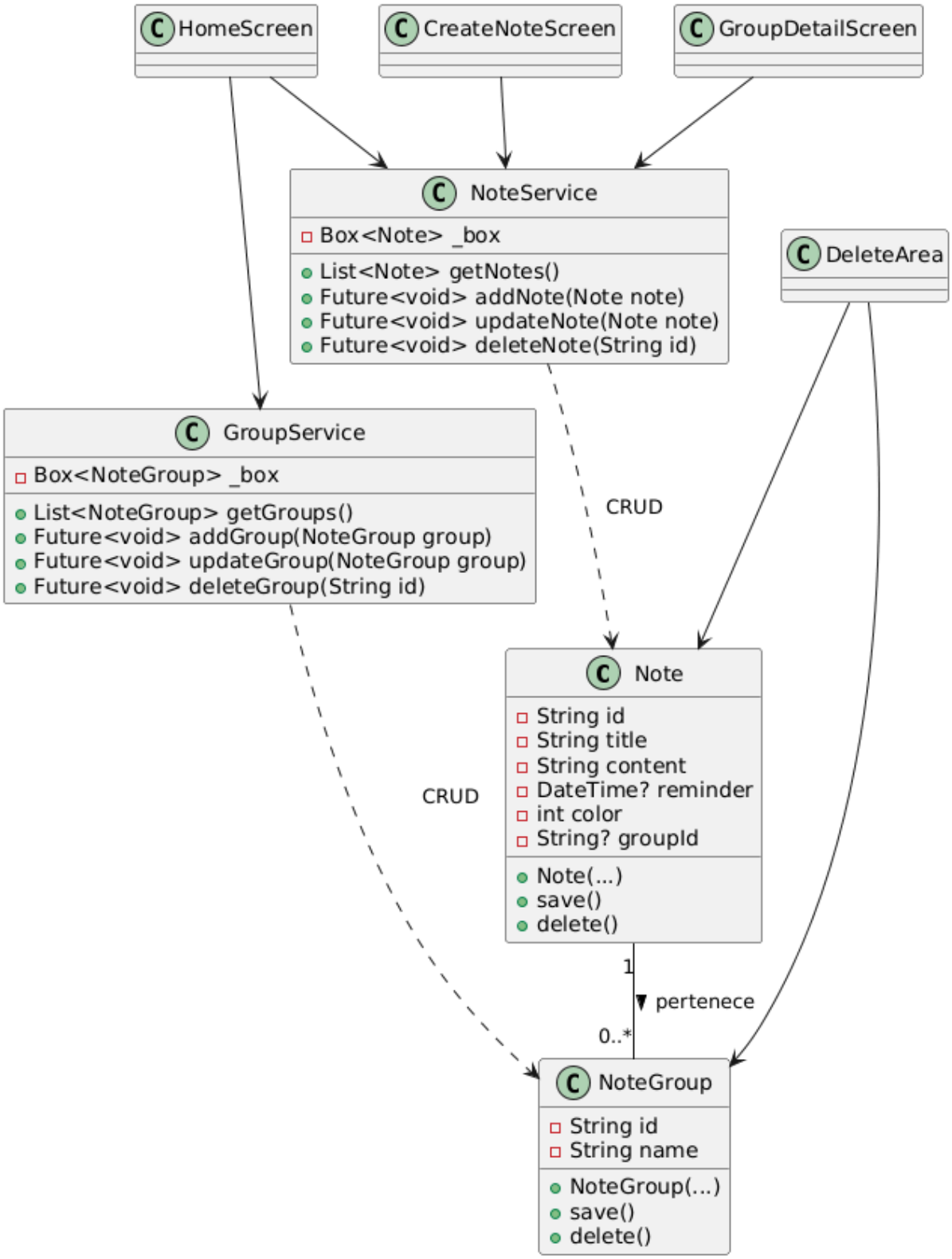

Figura 1: Diagrama de clases UML (generado con PlantUML).

3.2. Diagrama de flujo (agregar nota)

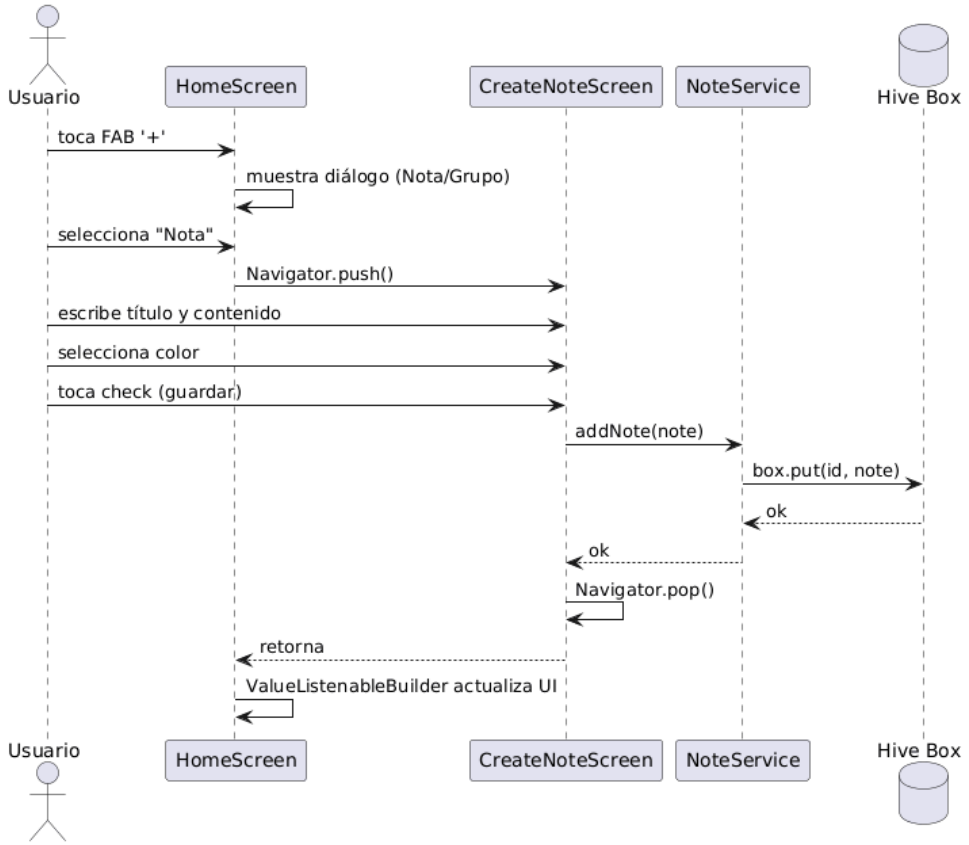

Figura 2: Flujo de creación de una nota desde el botón flotante hasta la persistencia en Hive.

4. Integración con OpenRouter (IA)

Para dotar a la aplicación de inteligencia, utilizamos la API de **OpenRouter**, un servicio que permite acceder a múltiples modelos de lenguaje avanzados.

4.1. Cómo funciona la integración:

1. Cuando el usuario guarda una nota, se llama automáticamente al servicio `AIService`.
2. Se envía el título y contenido de la nota junto con la fecha actual al modelo de IA.
3. La IA analiza el texto en lenguaje natural y responde en formato JSON estructurado.
4. El JSON contiene:
   - `has_date`: Si detecta una fecha u hora
   - `date_time`: Fecha y hora en formato YYYY-MM-DD HH:MM
   - `category`: Categoría de la nota
   - `suggested_action`: “alarma”, “crear_grupo” o “ninguna”
   - `explanation`: Explicación breve para el usuario

5. Según la sugerencia, la app muestra un diálogo elegante ofreciendo al usuario aceptar o ignorar la recomendación.

Esta integración permite que la aplicación entienda el contexto de las notas y ayude proactivamente al usuario.

5. Pruebas Realizadas con la IA

Se realizaron las siguientes pruebas para demostrar el funcionamiento del asistente inteligente:

1. **Reunión con el equipo de Flutter**
   Contenido: "Mañana a las 10 am en la sala de juntas..."
   Resultado: La IA detectó correctamente la fecha y sugirió configurar una alarma para el 26 de marzo a las 10:00.

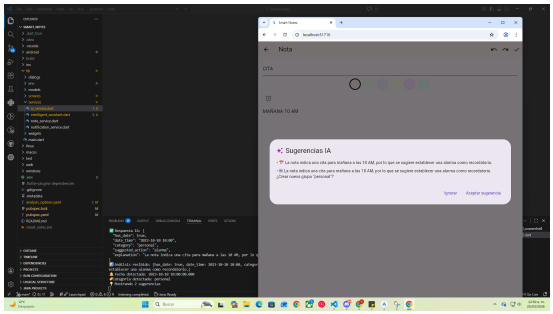

Figura 3: Sugerencia de alarma generada por la IA.

2. **Proyecto Flutter final**
   Contenido: "Terminar la integración de Hive... Fecha de entrega: próxima semana"
   Resultado: La IA sugirió crear un nuevo grupo llamado "trabajo".

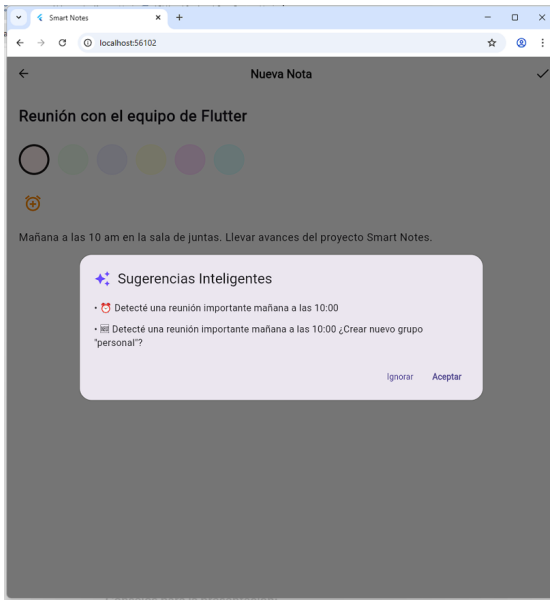

Figura 4: Sugerencia de grupo generada por la IA.

3. **Cita con el dentista**
   Contenido: “El viernes 27 de marzo a las 15:30...”
   Resultado: La IA detectó la cita y ofreció configurar un recordatorio.

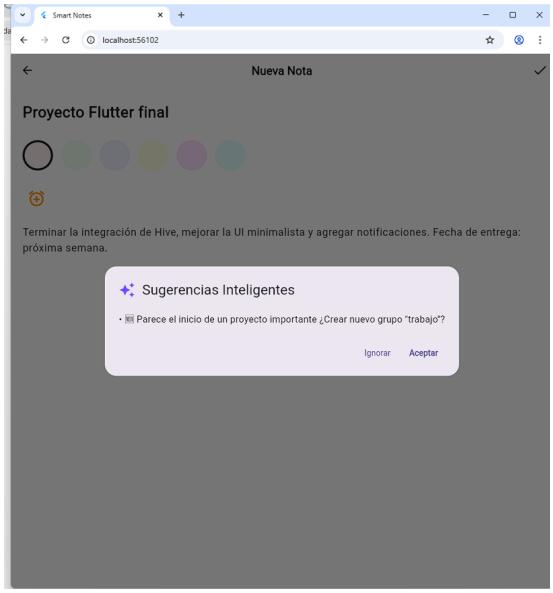

Figura 5: Sugerencia de recordatorio con fecha específica.

4. **Recordar comprar leche**
   Contenido: “Ir al supermercado esta tarde”
   Resultado: La IA sugirió configurar un recordatorio general.

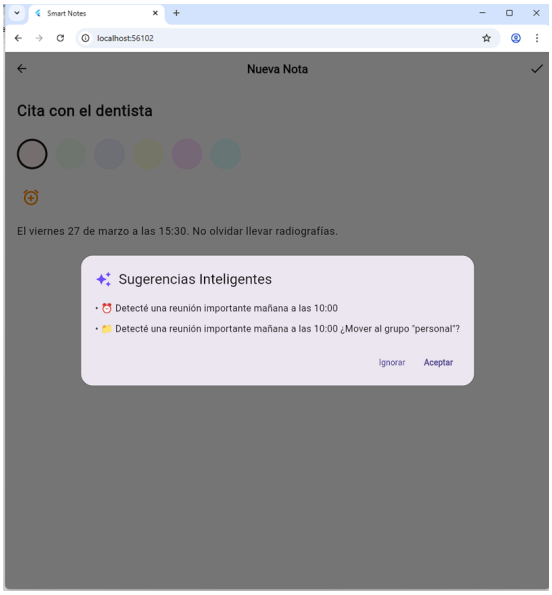

Figura 6: Sugerencia de recordatorio basada en horario.

6. Configuración Manual de Alarmas

Además de las sugerencias automáticas de la IA, la aplicación permite configurar recordatorios de forma manual:

- Al crear o editar una nota, el usuario puede presionar el icono de alarma.
- Se abre un selector de fecha y hora.
- Una vez guardada la nota, se programa una notificación local mediante `flutter_local_notificati`
- El usuario puede eliminar o modificar el recordatorio en cualquier momento.

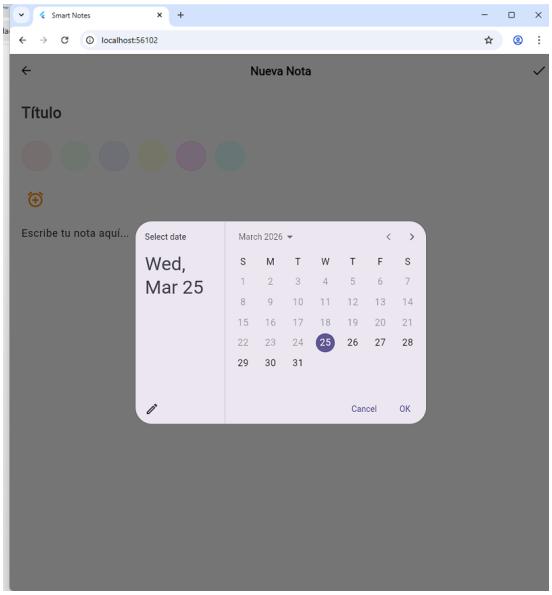

Figura 7: Pantalla de configuración manual de alarma.

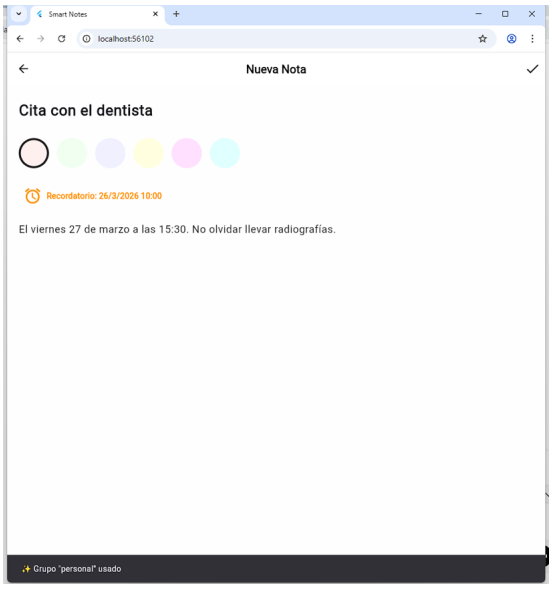

Figura 8: Ejemplo de alarma configurada en una nota.

7. Heurísticas de Nielsen: Evaluación del Proyecto

A continuación se analiza cómo cumple la aplicación con las 10 heurísticas de usabilidad de Jakob Nielsen:

1. **Visibilidad del estado del sistema:** Cumple. La aplicación muestra siempre el estado actual mediante `ValueListenableBuilder`.
2. **Relación entre el sistema y el mundo real:** Cumple. Usa lenguaje cotidiano e iconos reconocibles.
3. **Control y libertad del usuario:** Cumple. Permite deshacer/rehacer, cancelar diálogos y navegar hacia atrás.
4. **Coherencia y estándares:** Cumple. Sigue las convenciones de Material Design.
5. **Prevención de errores:** Cumple parcialmente. Valida título vacío y pide confirmación antes de eliminar.
6. **Reconocimiento en lugar de recuerdo:** Cumple. Todas las acciones están visibles.
7. **Flexibilidad y eficiencia de uso:** Cumple. Permite arrastrar y soltar para usuarios avanzados.
8. **Diseño estético y minimalista:** Cumple. Interfaz limpia con colores pastel.
9. **Ayuda a los usuarios a diagnosticar errores:** Cumple parcialmente.
10. **Ayuda y documentación:** No cumple. Falta una pantalla de ayuda o tutorial.

8. Metas Cumplidas y Comparación

Se cumplieron las siguientes metas principales:

- Sistema completo de notas con persistencia local usando Hive.
- Gestión avanzada de grupos (creación, edición, eliminación y asignación de notas).
- Interacciones táctiles avanzadas: arrastrar y soltar (nota sobre nota, nota sobre grupo, eliminación por arrastre).
- Personalización visual (6 colores pastel).
- Historial de edición (Undo/Redo).
- Integración exitosa con OpenRouter para sugerencias inteligentes de alarmas y grupos.
- Notificaciones locales programables.

Las funcionalidades de arrastrar y soltar y la integración con IA superaron las expectativas iniciales.

9. Áreas que Quedaron Pendientes y Mejoras Futuras

Aunque se avanzó significativamente, quedaron pendientes las siguientes funcionalidades:

- Reconocimiento de texto mediante OCR (foto → nota).
- Reconocimiento de voz (dictado).
- Búsqueda global de notas.
- Manejo robusto de errores de red y límites de la IA.
- Sincronización en la nube.

10. Conclusión y Aprendizaje

El desarrollo de **Smart Notes** ha sido una experiencia muy enriquecedora. Logré integrar tecnologías modernas como Flutter, Hive y OpenRouter, comprendiendo profundamente el ciclo completo de desarrollo de una aplicación móvil: desde el diseño de la arquitectura, la persistencia de datos, las interacciones táctiles avanzadas, hasta la integración de inteligencia artificial.

Aprendí la importancia de una buena experiencia de usuario, la utilidad de las heurísticas de Nielsen y cómo la IA puede transformar una aplicación sencilla en una herramienta realmente inteligente y útil.

Este proyecto me permitió consolidar conocimientos de programación móvil, diseño de interfaces y desarrollo de funcionalidades complejas, preparándome mejor para futuros desafíos en el campo de la Interacción Humano-Computadora.